

The key difference in GRPO vs PPO: the advantage

$$\text{advantage} = \frac{(\text{Reward} - \text{group mean})}{\text{group std}} \quad A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_n\})}{\text{std}(\{r_1, r_2, \dots, r_n\})}$$

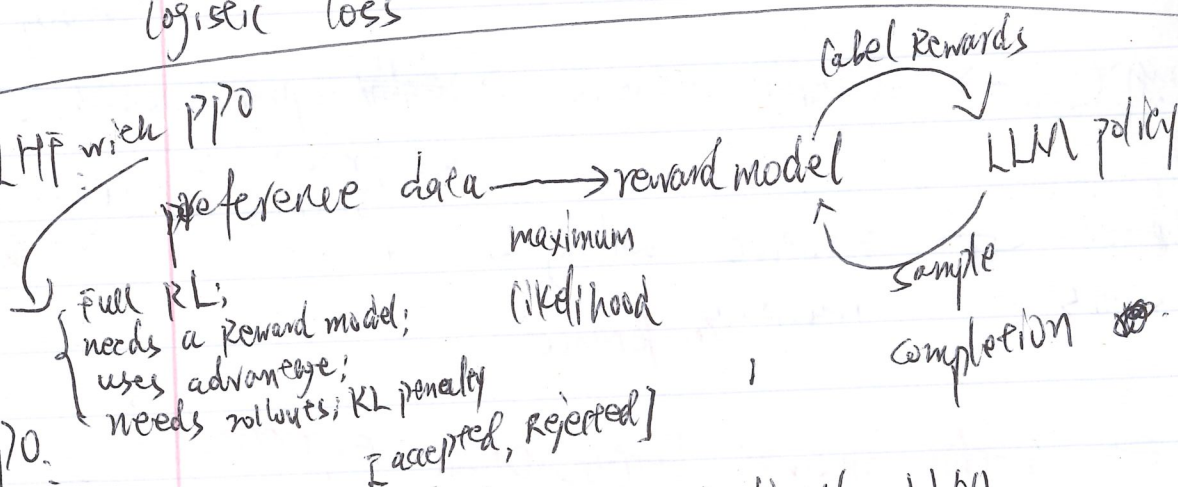
is this good? A minor RL detour

Baseline: we can subtract any state-dependent term from our rewards

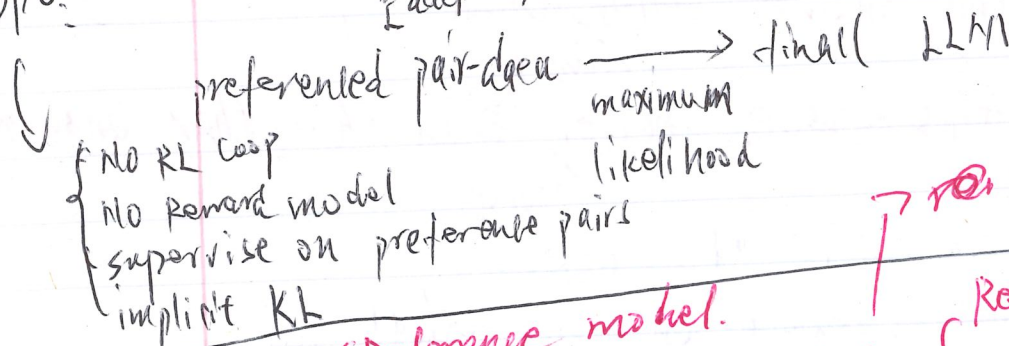
PPO: Reinforced learning with Reward model; uses RL on reward model KL penalty

DPO: supervised learning with preference pairs, use supervised learning on preference pairs (chosen/rejected) with logistic loss

RLHF with PPO



DPO:



no value model

PPO: { Reference model, Reward model, Value model }

GRPO: { Reference model, Reward model }

For GPTO, GPTO doesn't use a 'valid' baseline

⇒ the division by the ~~std~~ std term is not a valid baseline that preserves unbiasedness

⇒ what is an unbiased-gradient version of GPTO?

Also note the modification on the length-normalizer term on the left.

⇒ GPTO's advantage normalization introduce bias

$$\text{Advantage} = \frac{\text{Reward}_{\text{current}} - (\text{group-mean-rewards})}{\text{group-std-rewards}}$$

Notes:

In standard policy gradient methods (like REINFORCE or VPO), subtract a baseline [e.g. mean-advantage] from advantages reduces variance without introducing bias — as long as the baseline doesn't depend on the action.

⇒ introduce ^{biased} gradient
GPTO: estimates advantages by sampling multiple responses ('group') per prompt, then Normalizing Rewards within in the group, see equation above

However, std depends on all rewards in the group, including the current one
bias the gradient estimator
⇒ potential suboptimal convergence or infeasible Reinforce learning

Fix / solution: { Pr. GPTO: unbiased GPTO variant / unbiased gradient
Done Right: remove/adjust the std term, using only mean subtraction, or other normalization
 $\text{Advantage} = \text{Reward} - \text{mean}(\text{group-reward})$